RP GYM · Game Service

이벤트 기반 MSA에서 중복 · 순서 역전 · 유실을 견디는 데이터 정합성 설계

RP GYM은 걷기와 운동 기록으로 퀘스트를 깨면 경험치(XP)를 주는 건강 게임 앱이고, 저는 이벤트를 받아 퀘스트 완료를 판정하고 XP를 지급하는 Game Service를 맡았습니다.

Game Service의 핵심은 퀘스트 판정과 XP 지급이 중복이나 누락 없이 정확하고, 지급 결과를 신뢰할 수 있게 조회할 수 있고, 다른 서비스의 장애에 무너지지 않는 것입니다. 이 세 가지를 기준으로 나눠 작성했습니다.

팀 6명 · 4주 + 개인 리팩토링 **스택** Java 17 · Spring Boot 3 · JPA · PostgreSQL · Kafka · Redis **담당** 퀘스트 판정 · XP 지급 · 이벤트 수신 · 이벤트 발행

0장 구조 1장 쓰기 정합성 2장 읽기 모델 일관성 3장 장애 격리 부록 용량

장별 구성

장	다루는 것
0장 구조	전체 흐름 · 세 가지 핵심과 구조에서 정한 것
1장 쓰기 정합성	판정 · 지급 멱등성과 Lost Update · 파티 퀘스트 동시 완료
2장 읽기 모델 일관성	랭킹 덮어쓰기 방지 · 정합성 대조 · 랭킹 재구축
3장 장애 격리	Transactional Outbox · 예외 분류와 DLT 재처리 · 알림 · 읽기 실패가 쓰기를 되돌린 문제
부록	동기화 스파이크 처리 용량

퀘스트 판정, XP 지급 이력과 총 XP, 이벤트 수신과 Outbox 발행은 제가 맡아 설계부터 검증까지 진행했습니다.

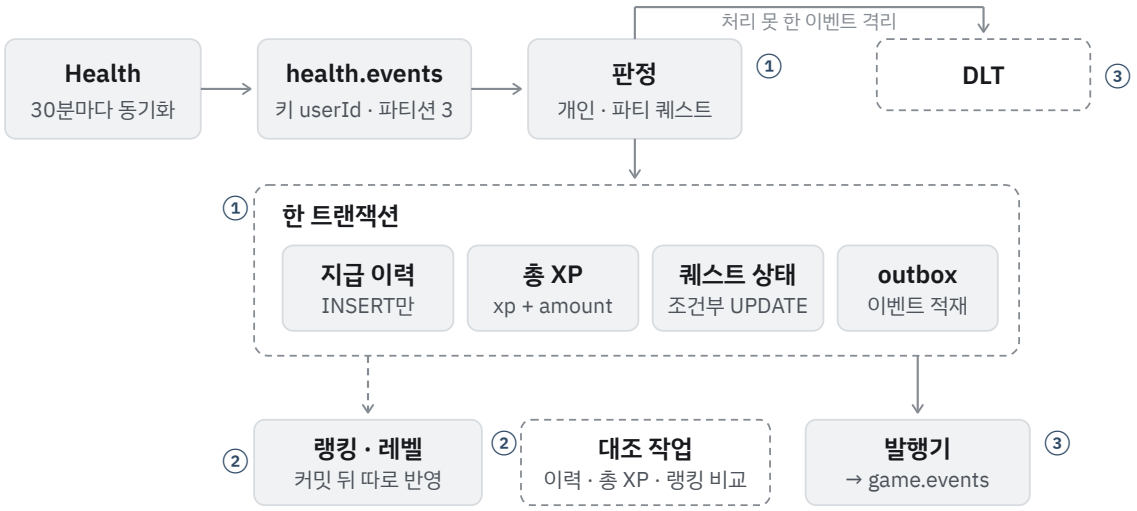
랭킹과 레벨 반영은 다른 팀원이 만든 것을 개인 리팩토링 때 고쳤고, 모니터링(Prometheus · Grafana · Loki)은 팀원이 구성한 위에 Game의 알림 규칙을 더했습니다.

0장

구조: 세 가지 핵심을 지키는 흐름

흐름을 정할 때는 누가 판정할지, 어디까지 한 트랜잭션으로 묶을지, 무엇을 커밋 뒤로 미룰지를 먼저 정했습니다.

전체 흐름



Health Service가 30분마다 걸음과 운동 기록을 Kafka 이벤트로 보내면, Game Service가 퀘스트 완료를 판정하고 XP 지급에 필요한 쓰기를 한 트랜잭션에서 커밋합니다.

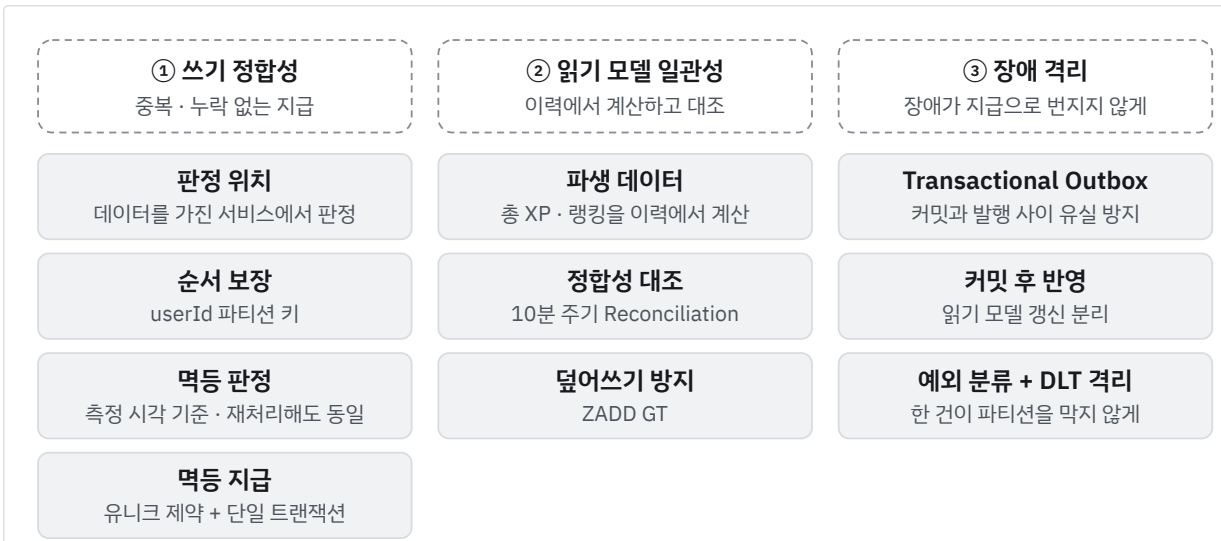
랭킹과 레벨은 커밋이 끝난 뒤 따로 반영하고, 다른 서비스에 알리는 이벤트는 outbox에 저장해 둔 것을 발행기가 꺼내 보냅니다.

그림의 ①②③은 각각 아래 세 가지 핵심을 지키는 부분입니다.

근거

- **테이블 설계**: ERD · V2 마이그레이션 · V7 마이그레이션

세 가지 핵심과 구조에서 정한 것



① 쓰기 정합성: 중복 · 누락 없는 지급

- **판정 위치**: 퀘스트 판정은 진행도와 기준값 데이터를 소유한 Game이 합니다. 따라서 판정할 때마다 다른 서비스를 조회할 필요가 없습니다.
- **순서 보장**: Kafka는 파티션 안에서만 순서를 보장하므로, 같은 사용자의 이벤트를 userId 키로 한 파티션에 모았습니다. 그래서 개인 퀘스트는 한 스레드가 순서대로 처리해 경합이 없습니다.
- **멱등 판정**: 서버 시각이 아니라 측정 시각을 기준으로 하고, 진행도는 누적값에서 기준값을 빼서 매번 다시 계산합니다. 순서가 역전된 이벤트나 다시 처리한 이벤트가 와도 결과가 같습니다.
- **멱등 지급**: 지급 이력(`xp_ledgers`)에 (사용자, 지급 출처 종류, 출처 ID) 유니크 제약을 걸고, 총 XP · 퀘스트 상태와 한 트랜잭션에서 커밋합니다. 같은 이벤트가 여러 번 와도 지급은 한 번이고, 일부만 반영된 상태가 남지 않습니다.

② 읽기 모델 일관성: 지급 이력에서 계산하고 대조

- **파생 데이터**: 총 XP(`wallets`)와 랭킹은 지급 이력에서 계산한 값이라, 틀어져도 지급 이력과 대조해 찾고 다시 만들 수 있습니다.
- **정합성 대조**: 10분마다 지급 이력 합계와 총 XP, 총 XP와 랭킹 점수를 비교해 막는 장치가 실제로 막았는지 확인합니다. 랭킹은 커밋 뒤 반영되므로, 다음 대조에서도 어긋나 있는 경우만 셉니다.
- **덮어쓰기 방지**: XP는 줄지 않으므로 랭킹은 더 큰 점수일 때만 반영해, 늦게 도착한 낮은 점수가 최신 점수를 덮지 않습니다.

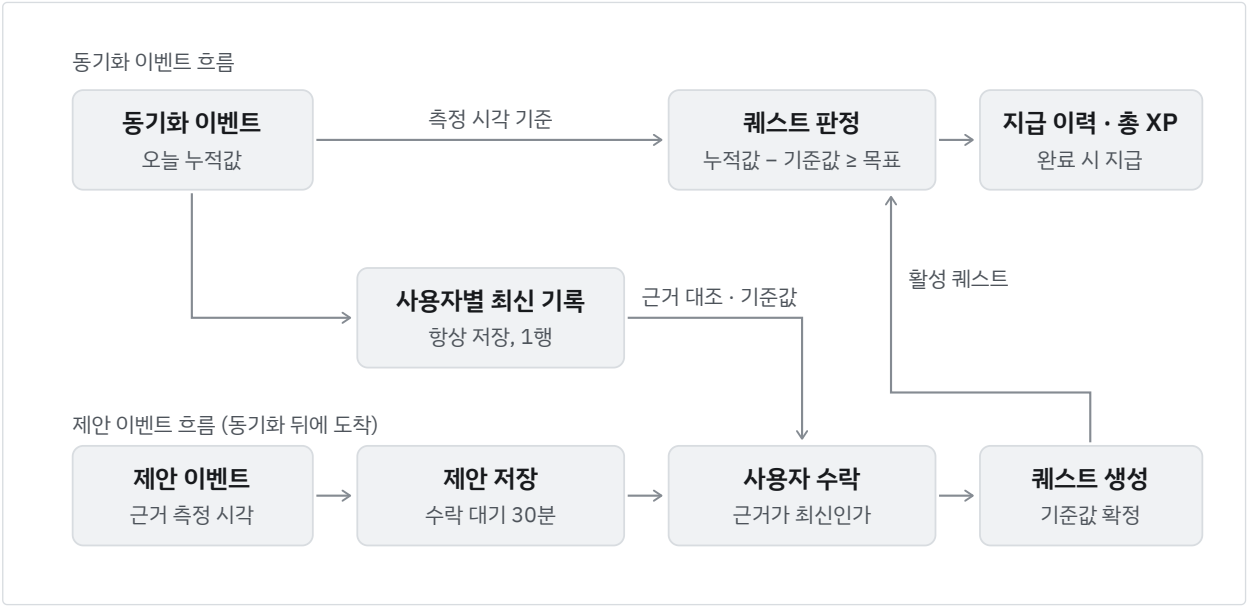
③ 장애 격리: 의존 서비스 장애가 지급으로 번지지 않게

- **Transactional Outbox**: 발행할 이벤트를 지급과 같은 트랜잭션에서 저장하고 발행기가 따로 보내므로, DB 커밋과 Kafka 발행 사이에 서버가 죽어도 이벤트를 잃지 않습니다.
- **커밋 후 반영**: 랭킹과 레벨은 지급 트랜잭션이 커밋된 뒤 따로 반영해, Redis 장애가 XP 지급까지 롤백시키지 않습니다.
- **예외 분류 + DLT 격리**: 일시 장애는 재시도하고 그 밖의 실패는 DLT로 격리해, 이벤트 한 건이 파티션 전체를 막지 않고 원인을 고친 뒤 다시 처리할 수 있습니다.

1장

쓰기 정합성: 중복 · 누락 없는 지급

판정 기준: 누적값 · 기준값 · 측정 시각



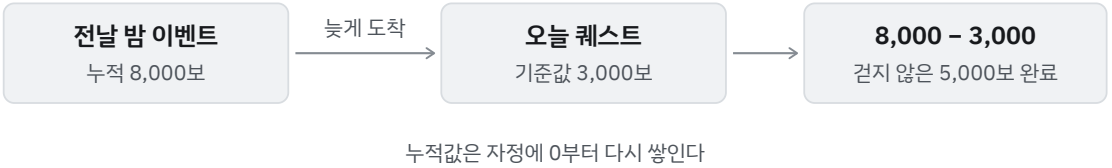
설계

- **누적값 − 기준값**: 진행도는 더하지 않고 이벤트마다 누적값에서 기준값을 빼서 다시 계산합니다. 같은 이벤트가 다시 오거나 중간 이벤트가 유실돼도 진행도가 맞습니다.
- **제안과 퀘스트 분리**: 수락 전 제안은 별도 테이블에 둬서, 퀘스트 테이블에는 기준값이 정해진 퀘스트만 남깁니다.
- **사용자별 최신 기록**: 제안에는 누적값이 없어 최신 기록을 따로 저장하고, 수락할 때 제안의 근거와 같은 기록일 때만 기준값으로 씁니다.
- **측정 시각 기준**: 만료와 순서는 서버 시각이 아닌 측정 시각으로 판단해, 언제 다시 처리해도 결과가 같습니다.

근거

- **코드**: Quest · QuestSuggestionService

순서 역전 방어: 측정 시각 워터마크 · 날짜 검사



문제

전날 이벤트가 늦게 도착하면, 오늘 걷지 않은 걸음으로 퀘스트가 완료될 수 있었습니다.

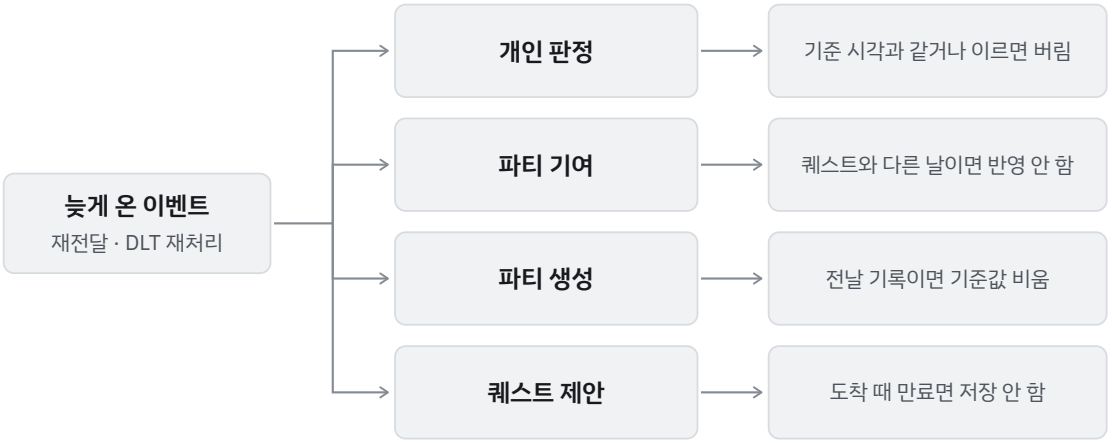
원인

누적값은 자정에 0으로 돌아가는데, 판정은 이벤트 날짜를 보지 않고 누적값에서 기준값을 빼기만 했습니다.

같은 사용자의 이벤트는 한 파티션에서 순서대로 처리돼 평소에는 드러나지 않았습니다.

하지만 재전달이나 DLT 재처리는 언제든 늦은 이벤트를 만듭니다.

조치



늦은 이벤트가 들어오는 네 경로를 각각 막았습니다.

- **개인 판정**: 기준값을 잡은 측정 시각과 같거나 이른 이벤트는 버립니다.
- **파티 기여**: 이벤트 날짜가 퀘스트 날짜와 다르면 반영하지 않습니다. 파티에는 기준값이 아직 없는 멤버가 있어서, 개인처럼 시각으로 거를 수 없었습니다.
- **파티 생성**: 멤버의 최신 기록이 오늘 것이 아니면 기준값을 비워 두고, 그 멤버의 오늘 첫 이벤트로 정합니다. 전날 값을 기준으로 잡으면 오늘 누적값이 그 값을 넘을 때까지 기여가 계속 무시됩니다.
- **퀘스트 제안**: 도착했을 때 이미 만료된 제안은 저장하지 않습니다. 저장하면 사용자가 누를 수 없는 알림 카드가 나갑니다.

검증

전날 이벤트와 31분 지난 제안을 두 경로로 넣었습니다. Kafka로 직접 넣는 경우와 DLT에서 재처리하는 경우입니다.

정상 이벤트가 그대로 반영되는지 보려고, 오늘 이벤트와 29분 지난 제안을 비교군으로 함께 넣었습니다.

결과

전날 이벤트만 받은 개인 퀘스트 25개는 보상 0건, 파티 퀘스트 25개는 전날 값이 기여로 잡힌 경우 0건, 만료된 제안 저장은 0건이었습니다.

DLT 재처리로도 추가 지급은 0건이었습니다.

이 방어가 있어서 3장의 DLT 재처리를 안심하고 할 수 있습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
주입	개인 퀘스트 25개 · 파티 퀘스트 25개에 전날 이벤트, 근거 시각이 31분 지난 제안
경로	Kafka 직접 주입 / DLT에 넣은 뒤 재처리
비교군	오늘 이벤트 · 29분 지난 제안

근거

- **늦게 온 이벤트 실험**: 결과 · 실험 폴더
- **DLT 재처리 실험**: 결과
- **코드**: Quest · PartyQuestMember · PartyQuestCreateService · QuestSuggestionService

XP 지급 멱등성과 동시성 제어: 유니크 제약 + 원자적 UPDATE



문제

같은 이벤트가 다시 오면 XP가 두 번 지급될 수 있었습니다.

또 개인 보상과 파티 보상이 같은 사용자에게 동시에 지급되면, 한쪽 증가분이 사라졌습니다.

원인

컨슈머가 처리 도중 죽으면 Kafka는 같은 이벤트를 다시 보냅니다.

파티 보상은 마지막으로 기여한 멤버의 스레드가 멤버 전원의 총 XP를 갱신합니다.

같은 순간 다른 멤버의 개인 보상이 자기 스레드에서 갱신되면, 두 트랜잭션이 같은 값을 읽고 각자 더해 썼습니다.

조치

- **멱등 지급**: 지급 이력에 (사용자, 지급 출처 종류, 출처 ID) 유니크 제약을 걸어, 같은 출처의 지급은 몇 번 와도 한 번만 저장됩니다. 지급 전 조회는 이미 지급한 이벤트를 예외 없이 건너뛰기 위한 것이고, 동시에 들어온 중복은 유니크 제약이 막습니다.
- **원자적 UPDATE**: 총 XP는 읽지 않고 `xp = xp + :amount` 한 문장으로 더합니다. 뒤 트랜잭션은 앞 트랜잭션이 끝날 때까지 기다렸다가 바뀐 값 위에 더합니다.

검증

멱등성은 이미 처리한 이벤트 10건을 DLT에서 다시 처리해 확인했습니다.

Lost Update는 400명에게 개인·파티 보상 800건을 동시에 지급해 확인했습니다. 겹침을 늘리려고 실험용 지연 200ms를 넣었고, 총 XP만 예전 방식(읽고 더하고 쓰기)으로 되돌린 비교군을 함께 돌렸습니다.

결과

재처리로 추가 지급된 XP는 0이었습니다.

비교군에서는 33명이 XP 990을 잃었고(모두 파티 보상 30), 지금 코드에서는 0명이었습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
멱등성	이미 처리한 이벤트 10건을 DLT에 넣고 재처리
Lost Update	400명 × 개인 +10 · 파티 +30 = 지급 800건, 실험용 지연 200ms
비교군	총 XP만 읽고 → 더하고 → 쓰기로 되돌린 버전
판정	총 XP와 지급 이력 합계가 다른 사용자 수

근거

- **멱등성 실험 (DLT 재처리)**: 결과
- **Lost Update 실험**: 비교군 결과 · 지금 코드 결과
- **코드**: XpGrantService · WalletJpaRepository

파티 퀘스트 완료 동시성 제어: 원자적 UPDATE



문제

파티 퀘스트는 멤버 기여를 한 행에 모아 완료를 판정합니다.

갱신이 겹치면 완료가 누락되거나 보상이 중복될 수 있었습니다.

원인

파티션 키가 userId라서, 같은 파티 멤버의 이벤트는 서로 다른 스레드에서 처리됩니다. 그래서 여러
스레드가 같은 파티 행을 동시에 갱신합니다.

- **완료 누락**: 각자 자기 기여만 더한 뒤 합계를 읽으면, 아직 커밋되지 않은 다른 멤버의 기여가 보이지 않아 아무도 완료로 판단하지 못합니다.
- **중복 지급**: 합계가 목표에 닿는 순간 여러 스레드가 모두 완료로 판단하면 보상이 여러 번 나갑니다.

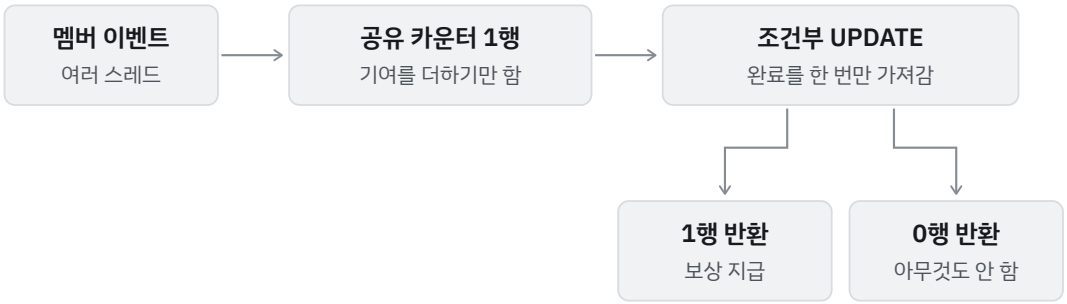
선택지

A. 카운터에 상태 조건 걸기 - 완료와 동시에 처리되던 기여가 멤버 기록에는 반영되고 카운터에서는 막혀, 카운터와 멤버 기여 합계가 어긋납니다.

B. 비관적 락 - 행을 잠그고 읽은 뒤 애플리케이션 레벨에서 판단해 다시 UPDATE하므로 DB 왕복이 두 번 이상입니다.

채택. 공유 카운터 + 조건부 UPDATE - 모든 멤버의 갱신이 한 행에서 차례로 처리되지만, 파티가 최대 4명이라 대기가 짧습니다.

조치



- **공유 카운터 (완료 누락)**: 멤버 기여를 `current_val = current_val + delta` 한 문장으로 더합니다. 뒤 스레드는 앞 스레드의 커밋을 기다렸다가 그 값 위에 더하므로, 마지막으로 더한 스레드는 전체 합계를 보고 판정합니다.
- **조건부 UPDATE (중복 지급)**: 진행 중이고 합계가 목표 이상일 때만 완료로 바꾸는 UPDATE 한 문장입니다. 먼저 실행한 스레드만 1행을 돌려받고, 보상은 그 스레드만 지급합니다.

검증

완료가 겹치는 최악 조건을 만들려고, 100개 파티(파티당 4명)의 이벤트를 파티션마다 같은 순번에 오도록 넣고 실험용 지연 200ms를 더했습니다.

같은 조건에서 완료 판단만 애플리케이션 레벨로 옮긴 비교군과, 컨슈머 스레드를 1개로 줄인 대조군도 돌렸습니다.

결과

99개 파티에서 여러 스레드가 동시에 완료를 시도했는데도, 100개 파티 모두 한 번만 완료돼 400명에게 보상이 한 번씩 지급됐습니다.

비교군에서는 28개 파티가 완료 누락, 9개 파티가 중복 판정됐습니다. 중복 판정은 지급 이력의 유니크 제약이 다시 막아 실제 중복 지급은 0건이었습니다.

실험용 지연으로 겹침 구간을 넓힌 스트레스 조건입니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
부하	파티 100개 × 멤버 4명 = 이벤트 400건, 목표 3,000
겹침 조건	파티 p의 멤버 i를 파티션 (p + i) % 3에 배치해 파티션마다 같은 순번, 실험용 지연 200ms
비교군	완료 판단을 애플리케이션 레벨로 옮긴 버전
대조군	컨슈머 스레드 1개

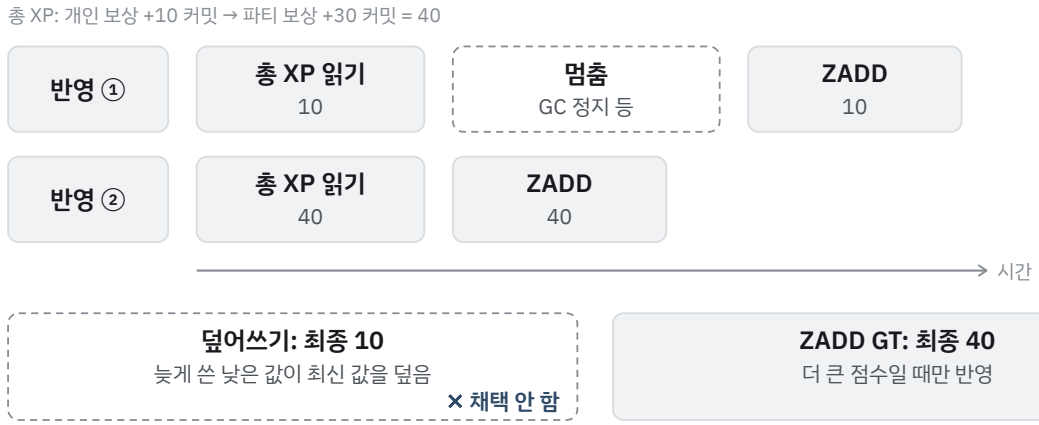
근거

- **A를 포기한 이유**: 경합 테스트
- **실경합 실험**: 실험 기록 · 비교군 결과
- **코드**: PartyQuestJpaRepository · PartyQuestProgressService

2장

읽기 모델 일관성: 지급 이력에서 계산하고 대조

랭킹 덮어쓰기 방지: ZADD GT



문제

늦게 반영된 낮은 점수가 최신 점수를 덮어, 랭킹이 총 XP보다 낮게 남을 수 있었습니다.

원인

랭킹은 지급이 커밋된 뒤 총 XP를 읽어 Redis에 씁니다.

같은 사용자의 지급 두 건이 각자 반영되면, 먼저 읽은 옛 값이 나중에 쓰일 수 있습니다. 기존 ZADD는 마지막에 쓴 값이 이깁니다.

조치

- **ZADD GT**: XP는 줄지 않으므로 랭킹은 더 큰 점수일 때만 반영합니다.
- **레벨**: 레벨도 `GREATEST`로 더 큰 값만 남깁니다.
- **대가**: 실제보다 큰 점수가 한 번 들어가면 재구축으로도 내려가지 않아, 그때는 키를 지우고 다시 만듭니다.

검증

400명에게 개인(+10) · 파티(+30) 보상을 지급하고, 옛 값을 읽은 반영만 2초 멈추게 해 GC 정지를 흉내 냈습니다.

같은 조건에서 덮어쓰기 버전을 비교군으로 돌렸습니다.

결과

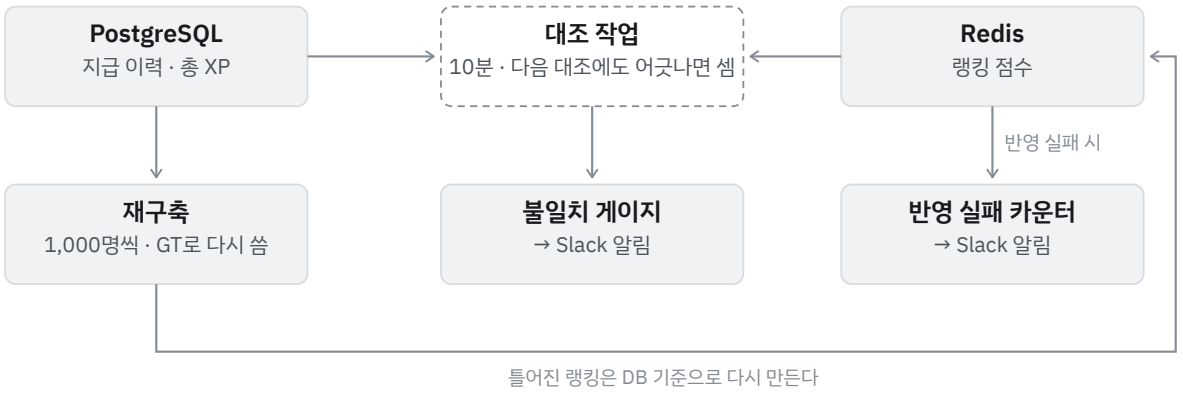
비교군은 10명의 랭킹이 총 XP보다 낮게 남았고, 대조 작업이 이 10명을 잡았습니다. 지금 코드는 0명입니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
부하	400명 × 개인 +10 · 파티 +30, 총 XP를 40 미만으로 읽은 반영만 2초 멈춤
비교군	ZADD 덮어쓰기 버전
판정	대조 작업의 랭킹 불일치 수

근거

- **랭킹 쓰기 순서 역전 실험**: 비교군 결과 · 지금 코드 결과
- **코드**: RankingRedisAdapter · CharacterRepository

랭킹이 틀어졌을 때 감지와 복구: 정합성 대조 · 실패 카운터 · 재구축



문제

랭킹은 커밋 뒤 따로 반영되므로, Redis가 멈추면 지급은 되고 랭킹만 빠집니다.

랭킹이 틀어져도 이를 감지하거나 복구할 수단이 없었습니다.

조치

- **반영 실패 카운터**: Redis 반영이 실패하면 지급은 그대로 두고 실패 횟수를 셉니다.
- **정합성 대조**: 10분마다 총 XP로 계산한 점수와 Redis 점수를 비교합니다. 한 번 어긋난 것은 반영이 잠깐 늦은 것일 수 있어, 다음 대조에서도 어긋나 있는 사용자만 셉니다.
- **재구축**: DB의 총 XP로 랭킹을 다시 만듭니다. 재구축 역시 GT 방식이라서 그 사이 들어온 더 큰 점수를 덮지 않습니다.

검증

Redis를 멈춘 채 400건을 지급하고, Redis를 다시 켠 뒤 재구축했습니다.

결과

지급 400건은 모두 성공했고 반영 실패 카운터가 400이 됐습니다.

Redis 복구 후 대조 작업이 불일치 525명(이번 400명과 이전 실험에서 남은 사용자)을 잡았고,
재구축 한 번으로 0이 됐습니다.

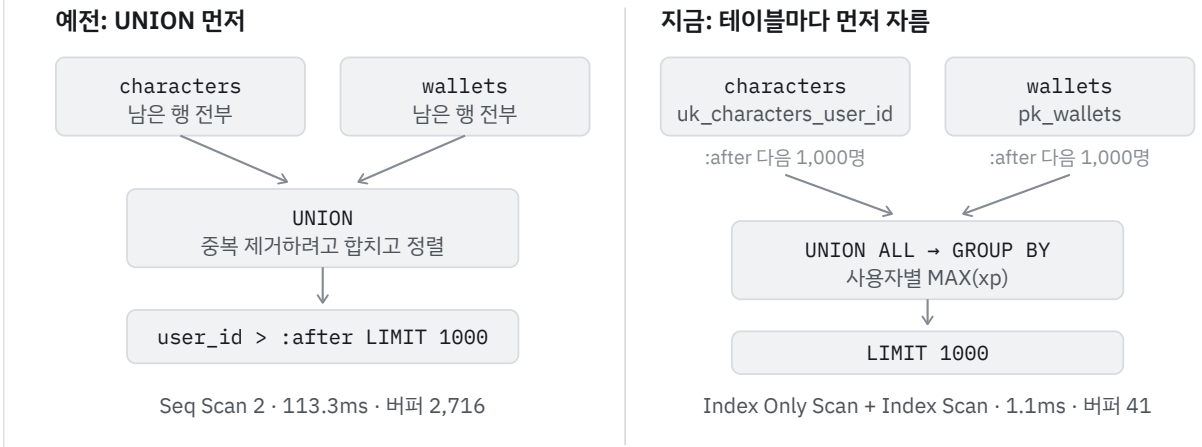
이 실험에서 Redis가 멈추면 대조 작업 전체가 멈춰 DB 검사까지 꺼지는 문제를 찾아, Redis 검사와
DB 검사를 분리했습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
순서	Redis 정지 → 이벤트 400건 지급 → Redis 재시작 → 대조 → 재구축 1회 → 대조
대조 주기	실험용 15초 (운영 10분)

근거

- **Redis 정지 중 지급 실험**: 결과
- **알림 실험 캡처**: Grafana 대조 불일치 캡처 · Grafana 랭킹 반영 실패 캡처 · Slack 대조 멈춤 알림
- **코드**: ReconciliationChecks · ReconciliationJob · RankingRebuildJob

10만 명 랭킹 재구축 시간을 69.7초에서 1.7초로 단축



문제

10만 명 기준 재구축이 69.7초 걸렸습니다.

원인

사용자마다 총 XP를 따로 조회했고(N+1), SQL 한 번으로 바꾼 뒤에도 UNION이 페이지마다 남은 행 전체를 합치고 정렬해 페이지당 113ms가 걸렸습니다.

조치

두 테이블에서 `user_id` 인덱스로 다음 1,000명만 읽고 합치게 쿼리를 바꿨습니다.

배치를 10,000으로 키우는 방법은 재구축 시간이 같은데 Redis 조회 꼬리 지연(p99.9)만 7.5배 늘어 쓰지 않았습니다.

검증

10만 명을 넣고 재구축하는 동안 다른 스레드에서 랭킹 조회 지연을 함께 쟀습니다.

같은 데이터에서 사용자별 조회 방식과 UNION 쿼리를 비교군으로 돌렸습니다.

결과

페이지당 113ms → 1.1ms, 재구축 69.7초 → 1.7초(40배). 재구축 중에도 랭킹 조회 p99는 2.3ms 이하였습니다.

환경	로컬 Docker · PostgreSQL 16 · Redis
데이터	10만 525명, 배치 1,000
실행 계획	5만 번째 사용자 다음 1,000명을 예전 쿼리와 지금 쿼리로 같은 실행 안에서 비교
조회 지연	재구축하는 동안 다른 스레드에서 랭킹 조회(ZSCORE) 지연 측정
비교군	사용자별 조회 방식 · UNION 쿼리

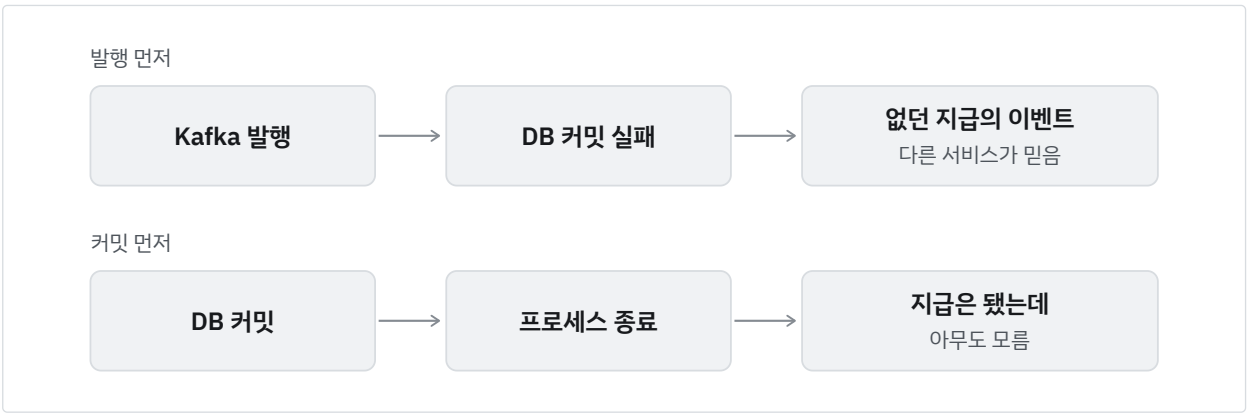
근거

- **재구축 실험**: 결과 · 실행 계획
- **코드**: CharacterRepository · RankingRebuildJob

3장

장애 격리: 의존 서비스 장애가 지급으로 번지지 않게

DB 저장과 이벤트 발행 사이 유실 방지: Transactional Outbox



문제

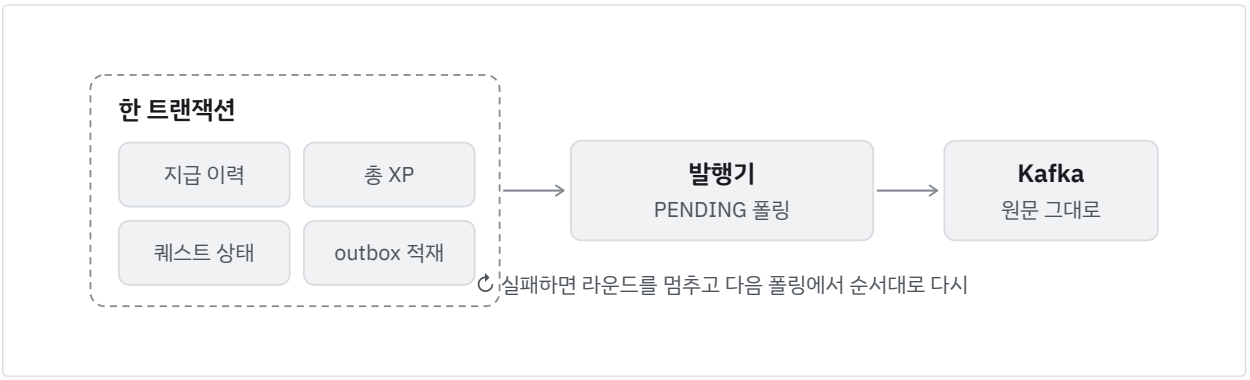
XP는 지급됐는데 완료 이벤트가 나가지 않거나, 지급이 롤백됐는데 이벤트만 나갈 수 있었습니다.

원인

DB와 Kafka는 서로 다른 시스템이라 한 트랜잭션으로 묶을 수 없습니다.

따라서 커밋한 뒤 발행 전에 서버가 죽으면 이벤트가 사라지고, 발행한 뒤 커밋이 실패하면 없던 지급의 이벤트가 나갑니다.

조치



- **같은 트랜잭션에 저장**: 발행할 이벤트를 지급 이력 · 총 XP · 퀘스트 상태와 함께 outbox 테이블에 저장합니다.
- **별도 발행기**: 대기 중인 이벤트를 꺼내 보내고, Kafka 응답을 받은 뒤에만 완료로 표시합니다.
- **실패하면 라운드 중단**: 발행 실패는 대부분 Kafka 브로커 장애 때문이라, 그 상태에서는 다음 이벤트도 보낼 수 없습니다. 그래서 그 라운드를 멈추고 다음 주기에 순서대로 다시 보냅니다.
- **원문 그대로 재발행**: 저장해 둔 이벤트를 그대로 보내서, 다시 보내도 eventId가 같아 받는 쪽이 중복을 거를 수 있습니다.
- **DLQ는 두지 않음**: DLQ도 같은 Kafka라 브로커 장애 때 같이 실패하고, 실패한 이벤트는 outbox에 대기 상태로 남아 있어 잃지 않습니다.

검증

outbox에 50건을 쌓은 뒤, 10건째를 보낸 직후 완료 표시가 커밋되기 전에 앱을 강제 종료하고 다시 켰습니다.

결과

재기동 후 50건 모두 발행돼 유실은 0건이었습니다.

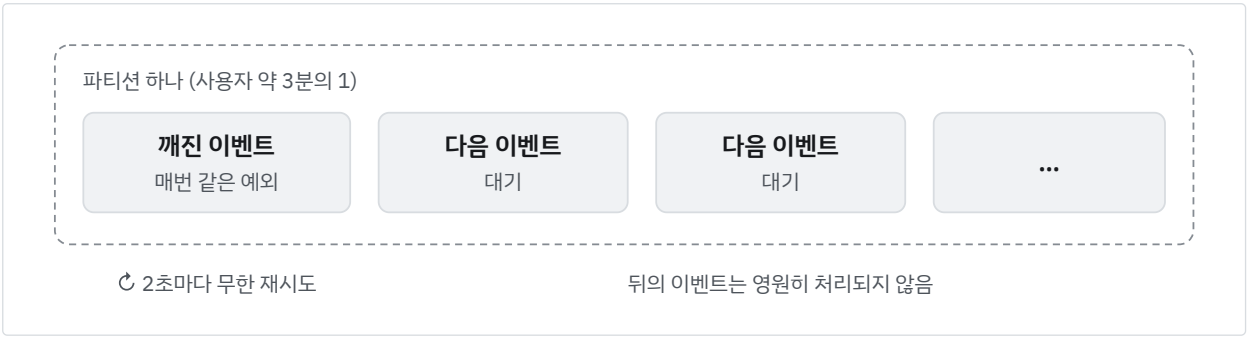
종료 전에 보낸 10건은 완료 표시가 커밋되지 않아 한 번 더 발행됐고(at-least-once), 같은 eventId라 받는 쪽에서 중복을 거를 수 있습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
부하	사용자 50명 · 이벤트 50건을 outbox에 쌓아 둠
장애	10건째 발행 직후, 완료 표시 커밋 전에 앱 강제 종료 → 재기동
세는 법	Kafka 브로커에서 레코드를 직접 덤프

근거

- **강제 종료 실험**: 결과 · Kafka 레코드
- **알림 실험 캡처**: Grafana Outbox 최장 대기 캡처
- **코드**: OutboxRelay · OutboxPublishScheduler

실패 이벤트 격리 및 재처리: 예외 분류 DLT + 재처리 횟수 제한



문제

매번 같은 예외가 나는 이벤트 한 건이 그 파티션의 처리를 멈췄습니다.

DLT로 격리한 뒤에는 원인을 고쳐도 그 이벤트를 다시 처리할 방법이 없었습니다.

원인

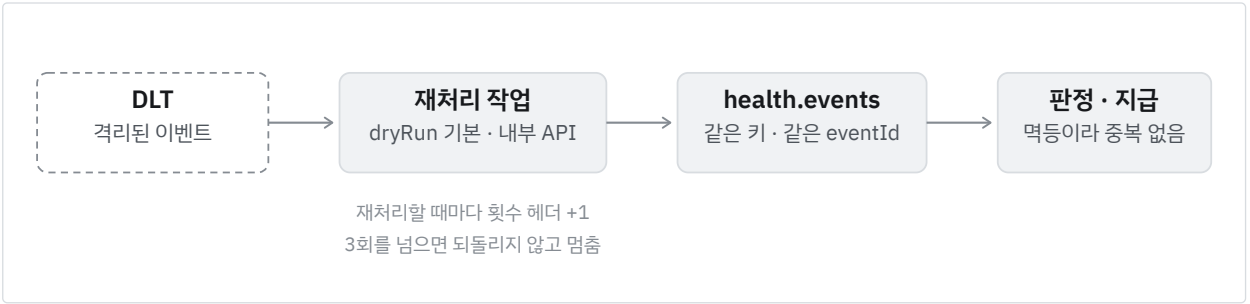
처음에는 재시도만 계속하면 유실을 막을 수 있다고 판단해 모든 실패를 무한 재시도했지만, 코드
결함 같은 예외는 재시도로 풀리지 않았습니다.

파티션 키가 userId라서, 그 파티션에 속한 사용자 약 3분의 1의 이벤트가 함께 멈춥니다.

조치



- **일시 장애**(DB · Redis 연결 실패, 낙관적 락 충돌 등): 성공할 때까지 재시도합니다. DLT로 보내면 장애 동안의 이벤트가 전부 DLT에 쌓이기 때문입니다.
- **스키마 위반**: 다시 해도 결과가 같아 바로 DLT로 보냅니다.
- **그 밖의 예외**: 1 · 2 · 4초 간격으로 세 번 재시도한 뒤 DLT로 보냅니다.
- DLT 발행에 성공한 뒤에만 다음 이벤트로 넘어가, 조용히 사라지는 이벤트가 없습니다.



- **재처리**: DLT의 이벤트를 같은 eventId로 원래 토픽에 다시 넣습니다. 판정과 지급이 멱등이라 다시 넣어도 중복 지급이 없습니다.
- **재처리 횟수 제한**: 횟수 헤더로 3회까지만 되돌려, 고쳐지지 않는 이벤트가 DLT와 원래 토픽을 계속 오가지 않게 했습니다.

검증

낙관적 락 충돌을 '그 밖의 예외' 재시도 한도보다 많은 5번씩 일으켜, 일시 장애로 분류되는지
확인했습니다.

버그로 DLT에 간 이벤트와 깨진 JSON을, 버그를 고친 뒤 재처리했습니다.

결과

낙관적 락 충돌 50번은 DLT 0건으로 모두 정상 처리됐습니다.

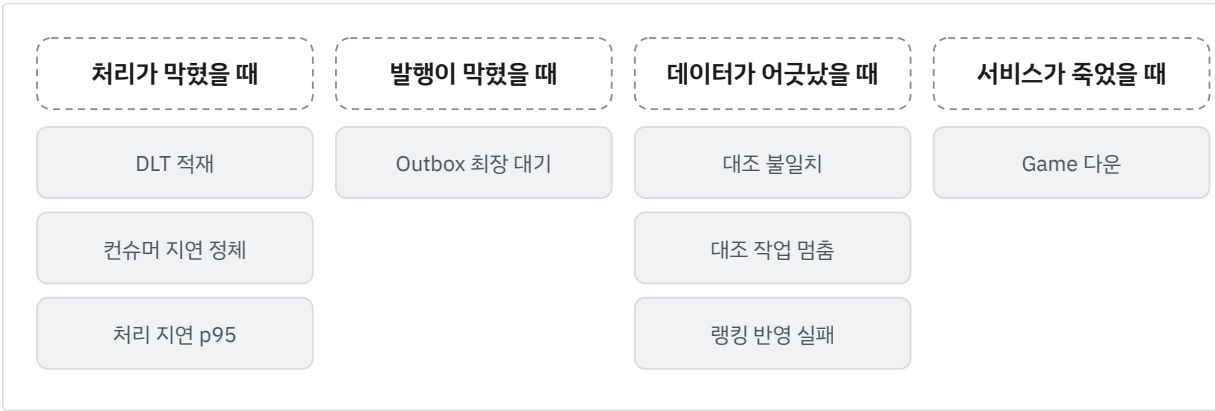
버그로 DLT에 갔던 10건은 재처리 뒤 보상이 지급됐고, 깨진 JSON은 3번 되돌린 뒤 멈췄습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3
예외 분류	퀘스트 10개 × 낙관적 락 충돌 5번 (그 밖의 예외 재시도 한도 3번보다 많게)
재처리	버그로 DLT에 간 이벤트 10건 + 깨진 JSON 1건, 버그를 고친 뒤 재처리

근거

- **낙관적 락 충돌 실험**: 결과
- **DLT 재처리 실험**: 결과
- **알림 실험 캡처**: Grafana DLT 적재 캡처 · Loki DLT 원인 로그
- **코드**: KafkaConsumerConfig · DltReplayService

모니터링: Game 알림 8종



구현

팀이 구성한 모니터링(Prometheus · Grafana · Loki · Slack) 위에 Game 알림 규칙 8종을 더했습니다.

격리하거나 반영이 실패해도 모니터링이 없으면 이유를 알 수 없어서, 처리 · 발행 · 데이터 · 서비스 상태를 각각 알리게 했습니다.

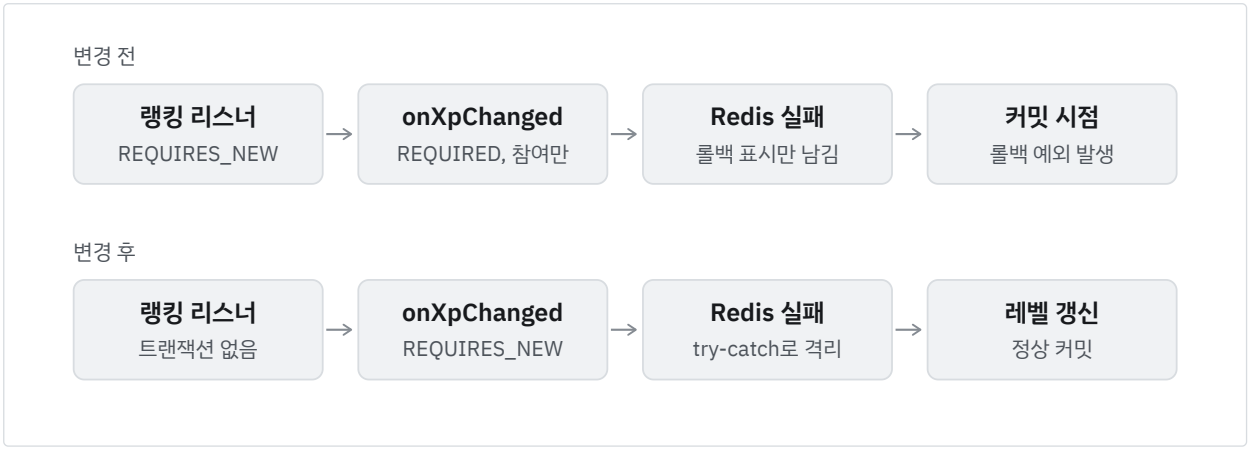
알림이 실제로 울리는지는 장애를 일부러 일으켜 확인했습니다.

환경	로컬 Docker · PostgreSQL 16 · Kafka 단일 브로커(파티션 3) · Redis · 컨슈머 스레드 3 + Prometheus · Grafana 알림 · Loki
방법	장애를 하나씩 일부러 일으키고 Slack 알림이 오기까지 시간 측정
감지 시간	DLT 60초 · Outbox 237초 · 컨슈머 지연 603초 · 대조 불일치 60초 · 랭킹 반영 실패 116초 · 처리 지연 593초 · 대조 멈춤 1,808초 · 서비스 다운 176초
오탐	평상시 30분 동안 0건

근거

- **일부러 장애를 일으킨 내용**: 글
- **알림 실험**: DLT · lag · Outbox · 대조 · 랭킹 · 처리 지연 · lag 오탐 · 다운 감지 · 알림 뒤 원인 추적
- **캡처**: lag 멈춘 파티션 · 처리 지연 p95 · lag 알림 · Slack 서비스 다운 알림
- **설정**: 알림 규칙

읽기 실패가 쓰기를 되돌린 문제: 트랜잭션 전파 설정 (협업)



문제

다른 팀원이 맡은 랭킹 리스너가 Redis 예외를 잡았는데도, 캐릭터 레벨 갱신이 롤백되고 있었습니다.

원인

Redis를 끄고 돌린 실험 로그에 예상하지 못한 예외 스택이 400건씩 쌓여 있어 추적했습니다.

catch로 예외를 잡은 5ms 뒤 커밋 시점에 `UnexpectedRollbackException`이 다시 났습니다.
서비스 메서드가 리스너의 트랜잭션에 참여만 해서, 예외가 나면 롤백 표시만 남고 커밋 때 전체가 롤백된 것입니다.

조치

- `REQUIRES_NEW`를 리스너에서 `onXpChanged`로 옮겨, 트랜잭션을 연 메서드가 예외를 직접 처리하게 했습니다.
- Redis 호출만 try-catch로 감싸, Redis 실패가 레벨 갱신까지 되돌리지 않게 했습니다.

결과

원인인 트랜잭션 전파 설정을 찾아 담당 팀원에게 전달했고, 당일 반영됐습니다.

같은 구조였던 업적 기능도 팀원에게 수정을 요청해 PR(#132)로 반영됐습니다.

XP 지급은 커밋이 끝난 뒤라 영향이 없었고, 레벨 갱신만 함께 롤백되고 있었습니다.

발견 경로	파티 동시성 실험(400건) 중 Redis를 띄우지 않은 상태에서 발견
관측	예외 스택 800줄 = RedisConnectionFailureException 400 + UnexpectedRollbackException 400
확정 근거	같은 이벤트의 로그 시각: catch 15:23:49.817 → 재발생 15:23:49.822
영향 범위	XP 지급 · 퀘스트 완료는 커밋 후라 안전, 레벨 갱신만 함께 롤백

근거

- **협업**: PR #132
- **코드**: RankingCommandService · XpGrantedEventListener

부록

용량: 이 구조로 얼마나 처리할 수 있는가

동기화 스파이크 처리 용량 측정: 커밋 횟수와 WAL 동기화 병목 규명

목적

30분마다 전체 사용자의 동기화가 한꺼번에 들어올 때, Game Service가 다음 정각 전에 몇 명까지 처리할 수 있는지 알고 싶었습니다.

앞선 실험 로그에서 이벤트 한 건에 0.6초가 걸린 것처럼 보여, 이 값이 맞는지도 확인해야 했습니다.

측정



- **Kafka에 직접 주입**: 동기화 API를 거치면 Health의 AI 호출(4~16초)이 먼저 병목이 되어 Game 컨슈머를 잴 수 없습니다.
- **두 끝만 측정**: 가장 가벼운 경우(퀘스트 없음)와 가장 무거운 경우(개인 + 4인 파티 진행)를 쟀습니다. 실제로 퀘스트를 진행하는 사용자 비율은 알 수 없어서, 두 경우를 각각 재서 실제 처리량이 그 사이에 있다고 볼 수 있게 했습니다.
- **쌓았다가 한 번에 처리**: 처음에는 부하 도구로 이벤트를 보내면서 쟀는데, 부하 도구가 먼저 느려져 Game의 처리량을 제대로 잴 수 없었습니다. 그래서 컨슈머를 멈춘 상태로 이벤트 2만 건을 먼저 쌓아 두고, 컨슈머를 다시 켜서 모두 처리하는 데 걸린 시간을 쟀습니다.

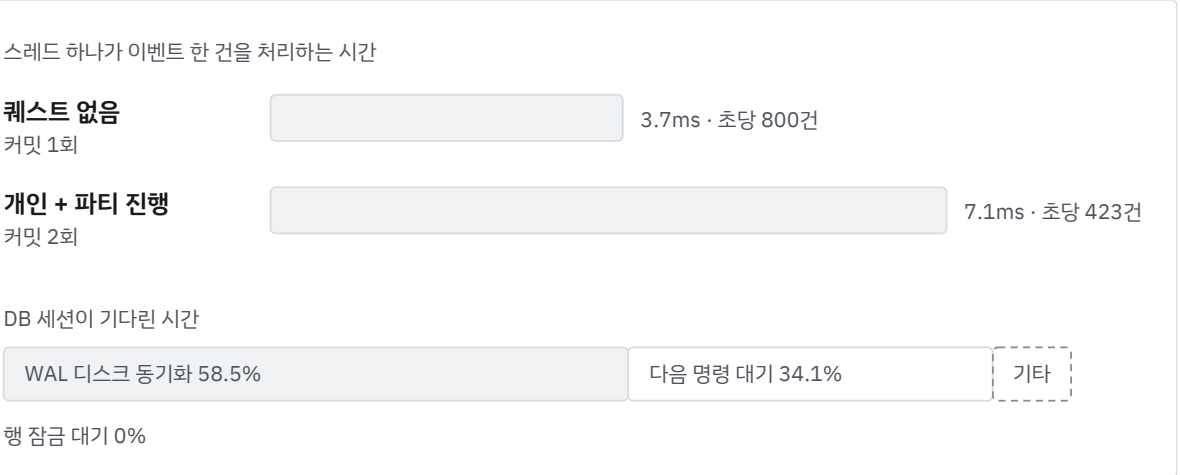
결과

처리량 · 가벼운 경우	처리량 · 무거운 경우	30분 처리 인원
800건/초	**423건/초**	**76만~144만**

가장 무거운 경우에도 초당 423건이라, 30분 한 주기에 76만 명까지 처리할 수 있습니다.

앞선 실험 로그로는 이벤트 한 건에 0.6초가 걸리는 것처럼 보였지만, 워밍업이 끝난 뒤 실제로 재 보니 3.7~7.1ms였습니다.

분석



건당 처리 시간이 커밋 횟수에 비례했고(1회 3.7ms, 2회 7.1ms), DB 세션 대기의 58.5%가 WAL 디스크 동기화, 행 잠금 대기는 0%였습니다.

커밋이 두 번인 이유는 개인 판정과 파티 판정을 다른 트랜잭션으로 나눴기 때문입니다. 파티에서 실패해도 개인 판정과 XP 지급까지 롤백되지 않게 나눈 것으로, 처리량 대신 장애 격리를 택한 트레이드오프입니다.

결론

병목은 확인했지만 개선은 적용하지 않았습니다. 현재 용량이 실제 사용 규모보다 훨씬 커서, 한계
용량과 병목 지점을 파악하는 데서 마무리했습니다.

환경	배포 서버 EC2 · 2 vCPU · 메모리 7.6GB · 컨테이너 19개 · Kafka 파티션 3 · 컨슈머 스레드 3
주입	JMeter(kafka-clients)로 `health.events` 에 직접 발행, 키는 userId, JMeter는 메모리 768MB, CPU는 워밍업 0.5코어 · 본 주입 1.5코어(컨슈머가 멈춰 있는 동안)로 제한
측정	컨슈머를 멈춘 채 2만 건을 쌓고 재개해 소진 시간만 측정, 워밍업 500건 선행, 진행은 DB 카운트로 관측
분해	DB 통계 증감 ÷ 20,000, 리스너 스레드 덤프 18회, DB 세션 대기 이벤트 샘플링, `docker stats`

근거

- **처리 용량 실험**: 측정 근거 · 실험 기록